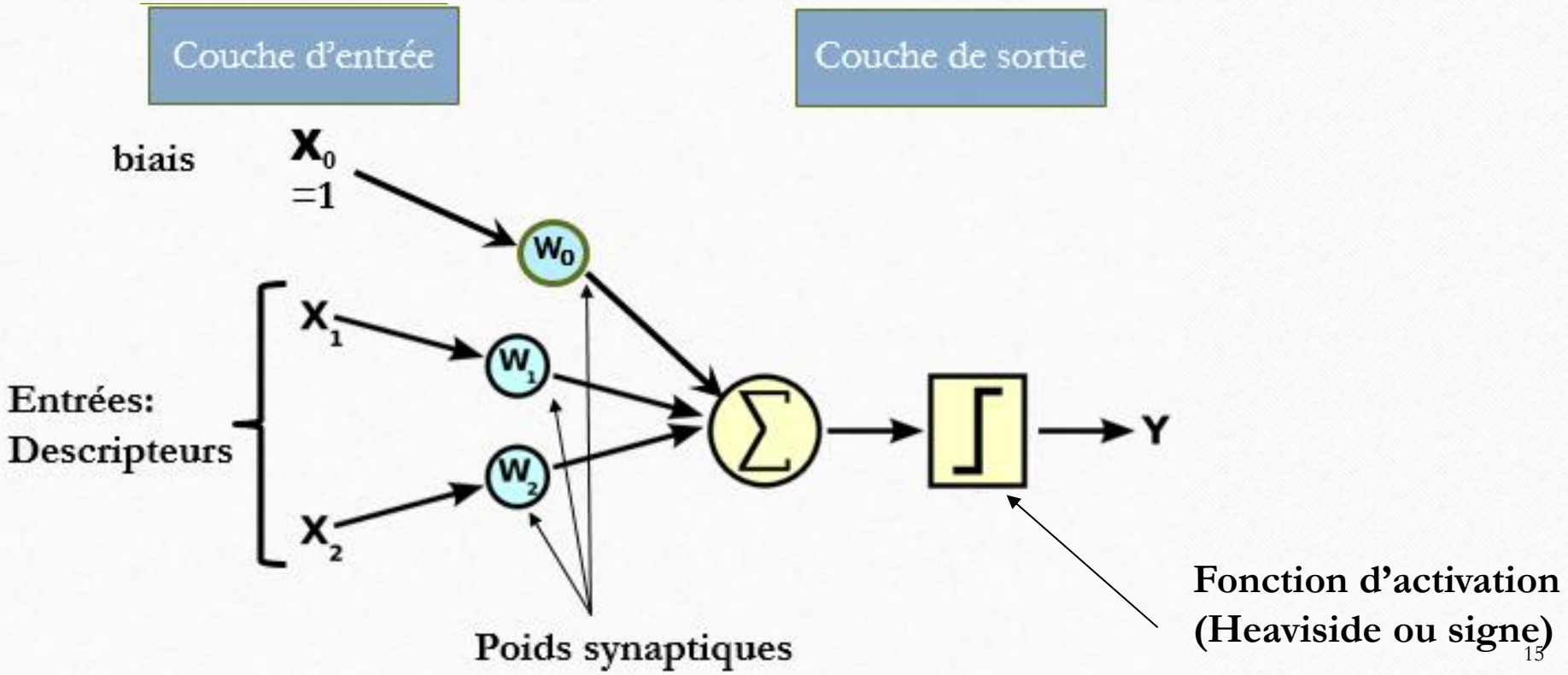
L'algorithme de perceptron

- Le perceptron est le modèle le plus simple de réseaux de neurones
- C'est un classificateur **linéaire**
- Le perceptron ne possède pas de couches cachées.

  → Une couche en entrée et une couche en sortie.

L'architecture de perceptron



L'algorithme de perceptron

1. Mélanger aléatoirement les observations
2. Initialiser aléatoirement les poids synaptiques
3. Faire passer les observations une à une
   - → Calculer l'erreur de prédiction pour l'observation
   - → Mettre à jour les poids synaptiques
4. Jusqu'à convergence du processus

L'algorithme de perceptron

- L'algorithme d'apprentissage doit adapter la valeur des paramètres (c'est-à-dire les poids et le biais) de façon que la sortie prévue $f_W(X)$ soit la bonne réponse sur les données d'entraînement.
- **Algorithme du Perceptron:**

1. Pour chaque paire $(X_t, Y_t) \in D$

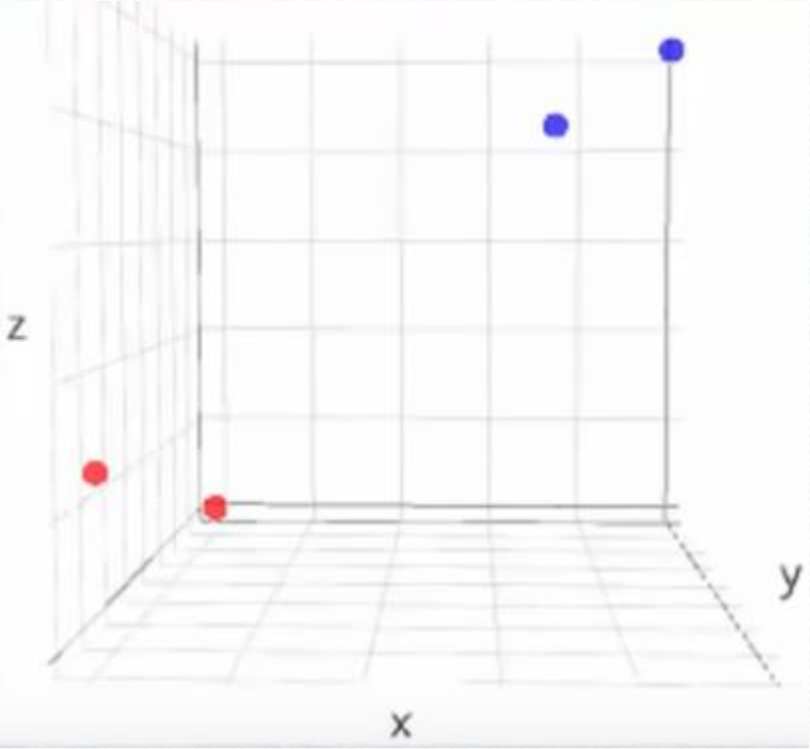
a. Calculer $f_W(X_t)$

b. Si $Y_t \neq f_W(X_t)$ alors $\mathbf{W_i \leftarrow W_i + \eta(Y_t - f_W(X_t))\, X_{t,i}}$ $\forall i$ (mise à jour des poids et du biais)

2. Retourner à 1 jusqu'à l'atteinte d'un critère d'arrêt (nb max d'itérations est atteint ou l'erreur = 0)

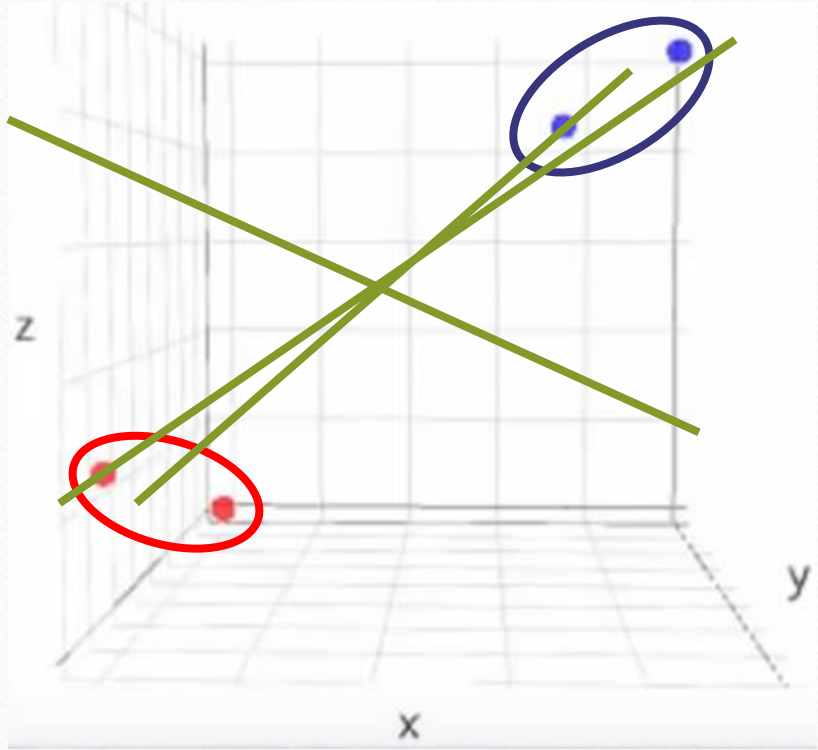
- La mise à jour des poids est appelée la règle d'apprentissage du perceptron.
- Le multiplicateur η est appelé le taux d'apprentissage.

Exemple de classification



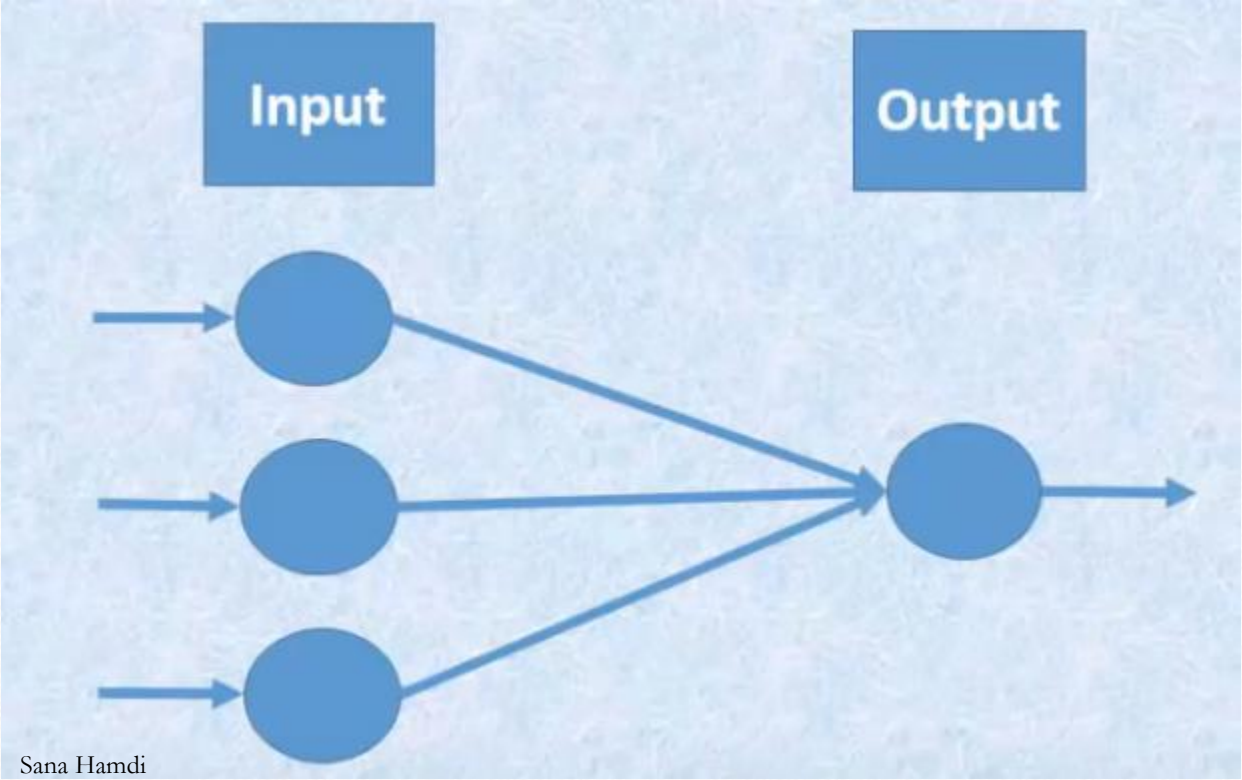
R (rouge)	G (vert)	B (bleu)	Classe
255	0	0	ROUGE
248	80	68	ROUGE
0	0	255	BLEU
67	15	210	BLEU

Exemple de classification

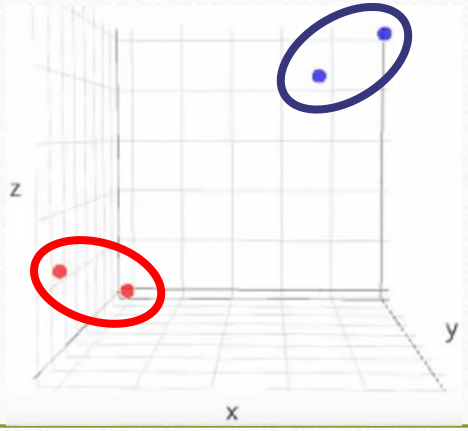


R (rouge)	G (vert)	B (bleu)	Classe
255	0	0	ROUGE
248	80	68	ROUGE
0	0	255	BLEU
67	15	210	BLEU

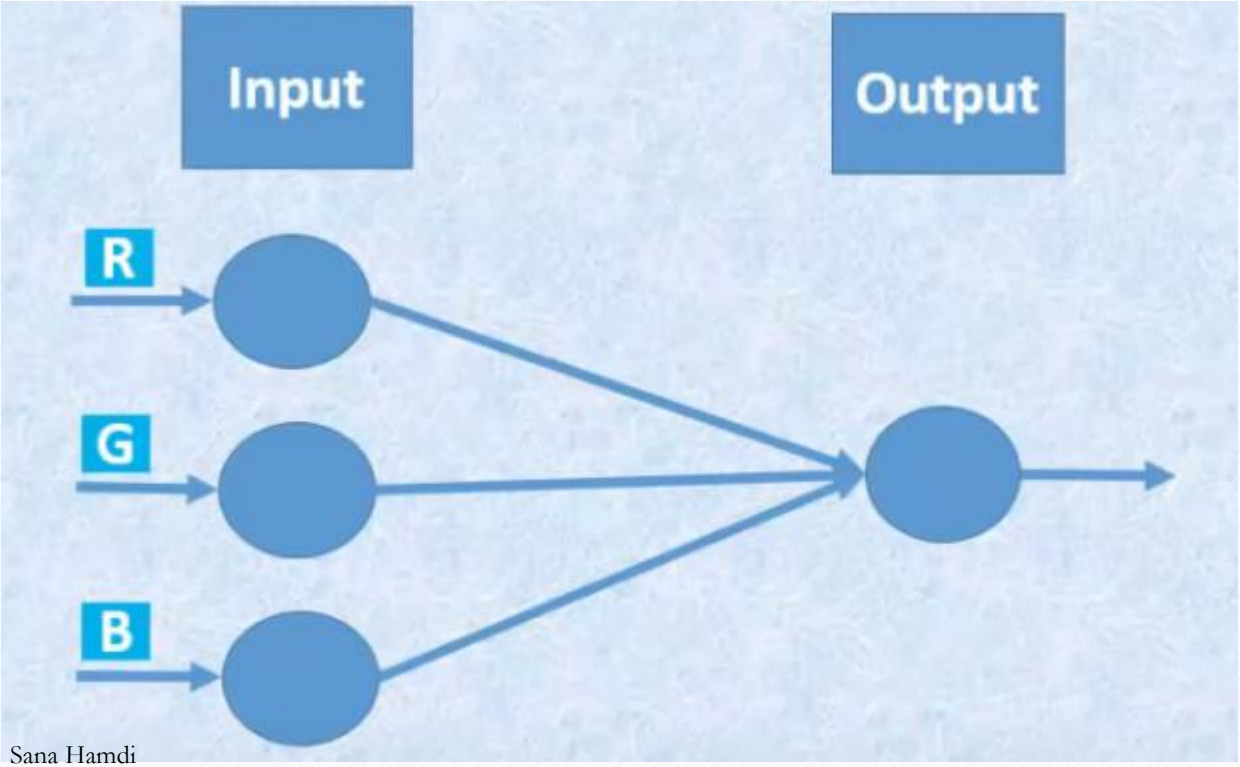
Exemple de classification



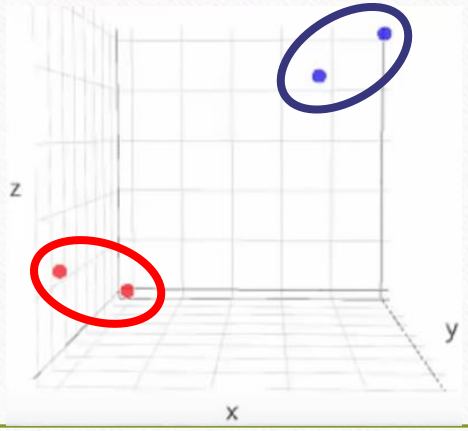
R (rouge)	G (vert)	B (bleu)	Classe
255	0	0	ROUGE
248	80	68	ROUGE
0	0	255	BLEU
67	15	210	BLEU



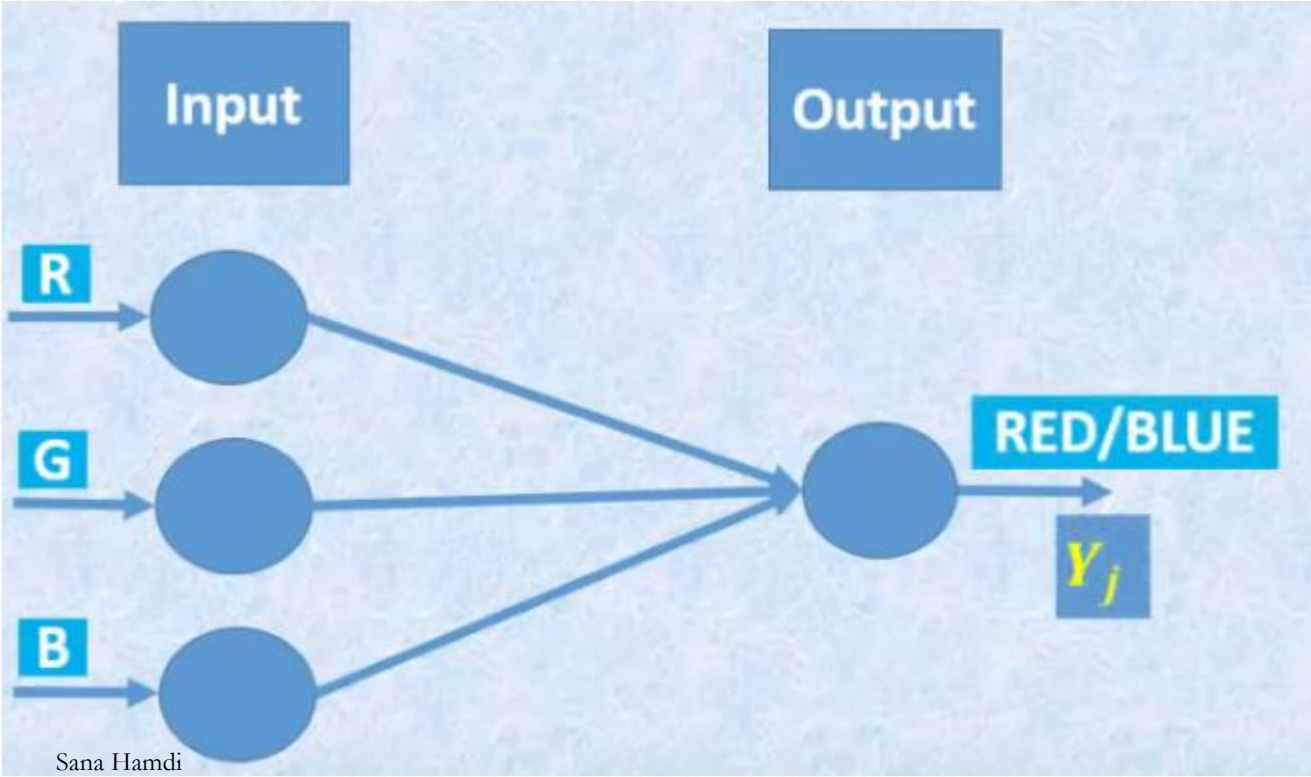
Couche d'entrée



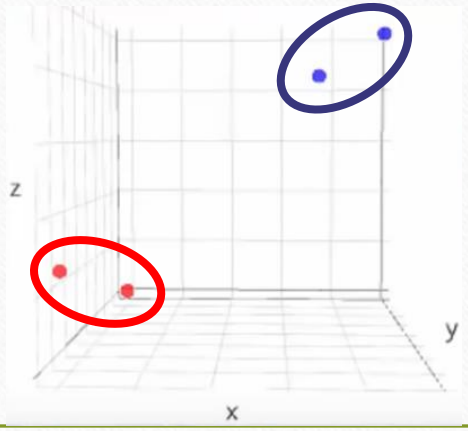
R (rouge)	G (vert)	B (bleu)	Classe
255	0	0	ROUGE
248	80	68	ROUGE
0	0	255	BLEU
67	15	210	BLEU



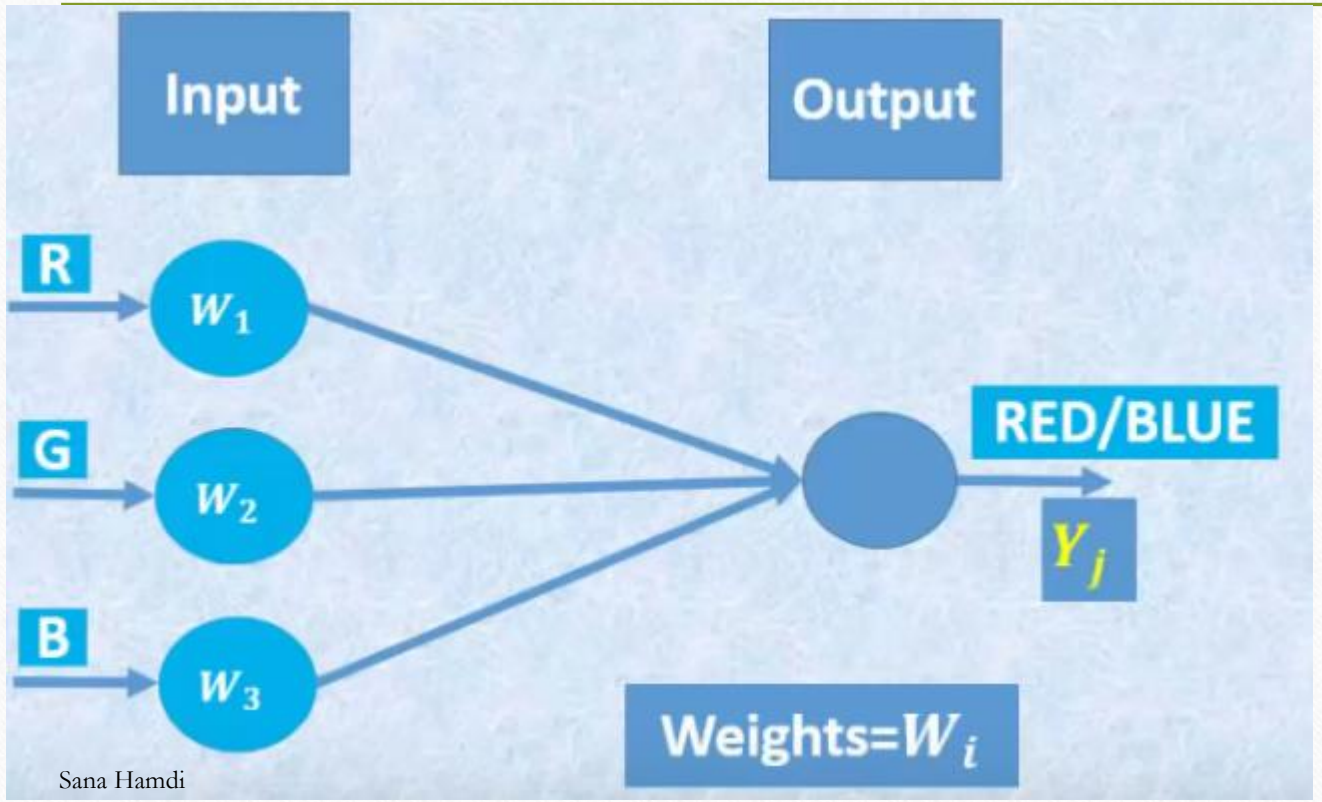
Couche de sortie



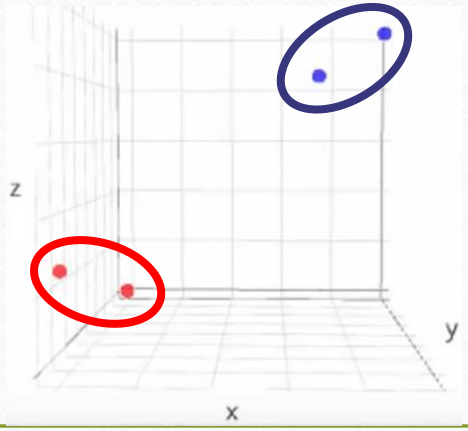
R (rouge)	G (vert)	B (bleu)	Classe
255	0	0	ROUGE
248	80	68	ROUGE
0	0	255	BLEU
67	15	210	BLEU



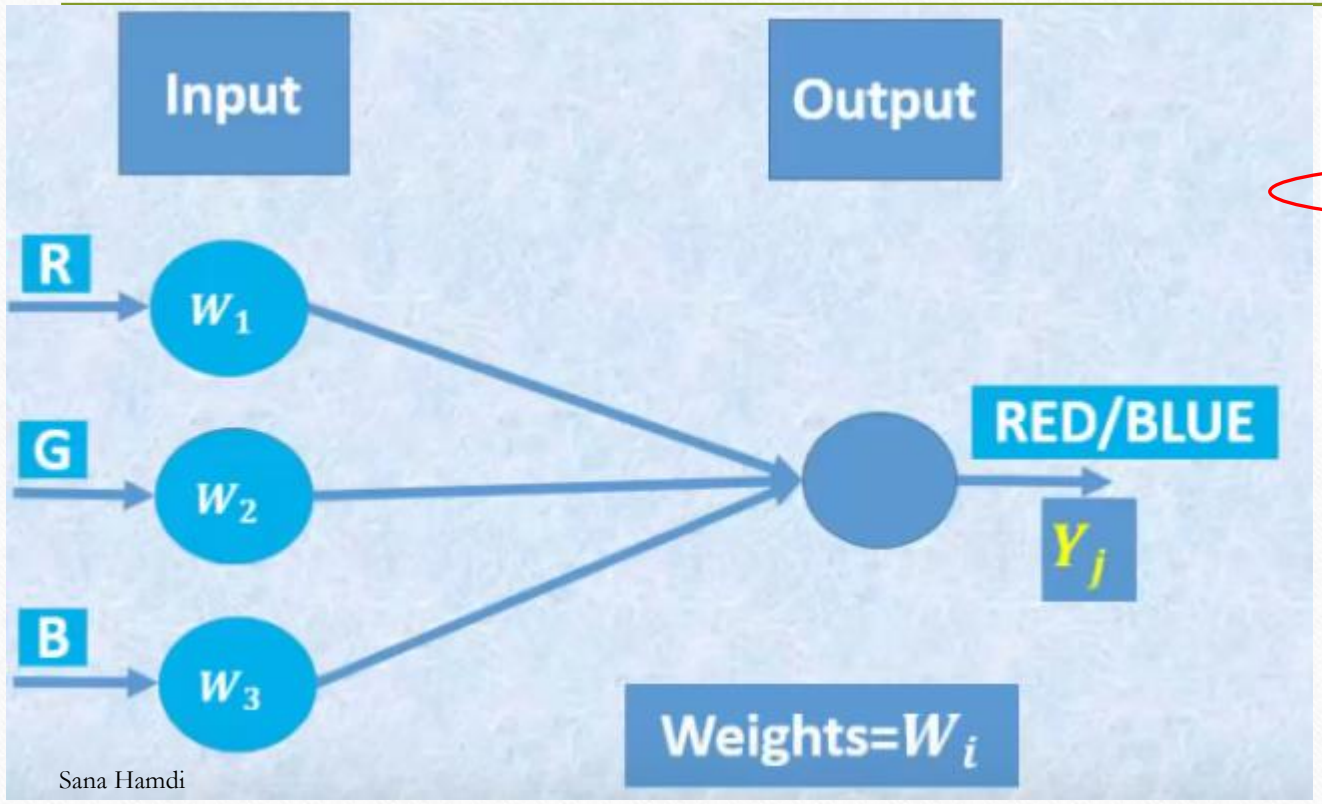
Les poids



R (rouge)	G (vert)	B (bleu)	Classe
255	0	0	ROUGE
248	80	68	ROUGE
0	0	255	BLEU
67	15	210	BLEU

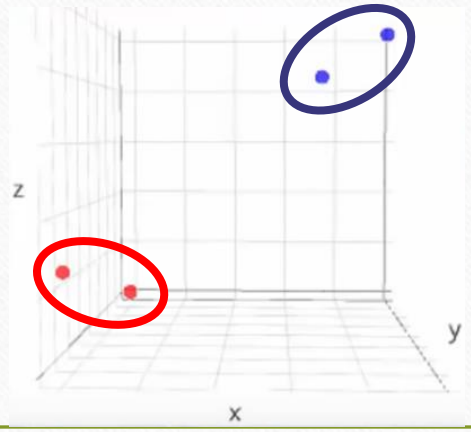


Les poids

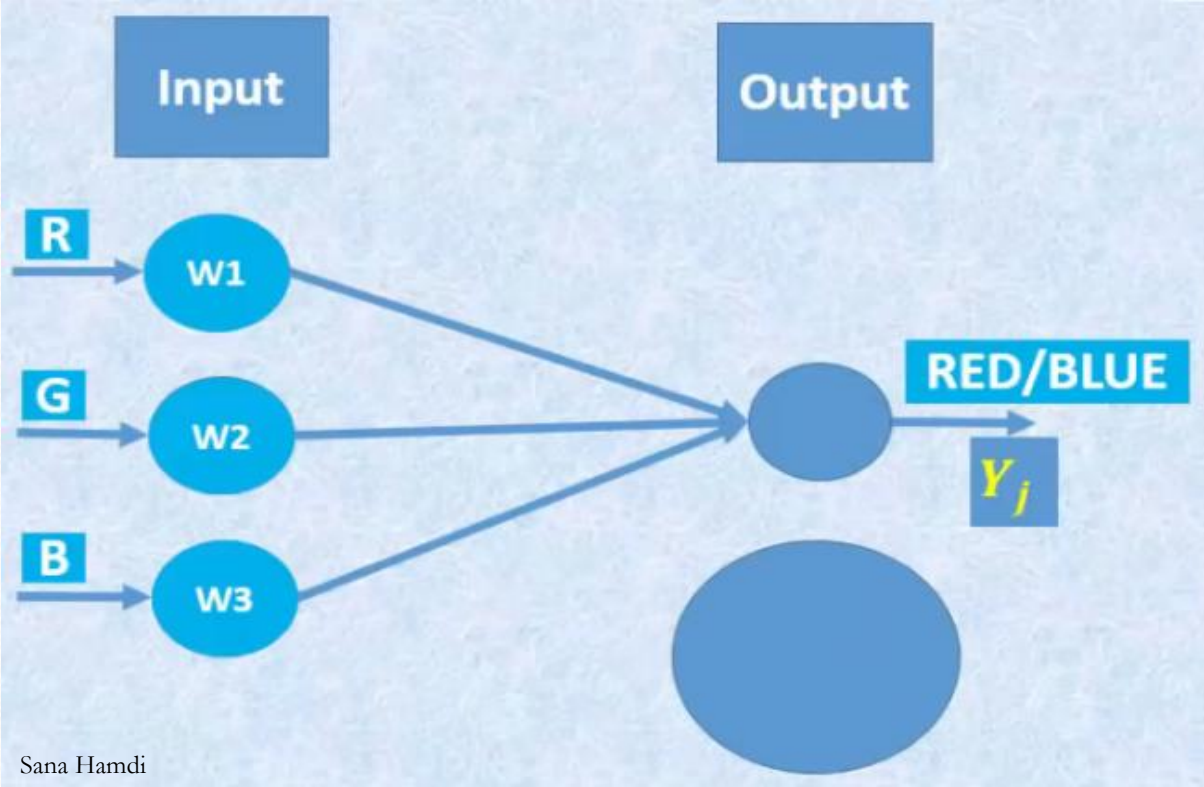


R (rouge)	G (vert)	B (bleu)	Classe
255	0	0	ROUGE
248	80	68	ROUGE
0	0	255	BLEU
67	15	210	BLEU

?

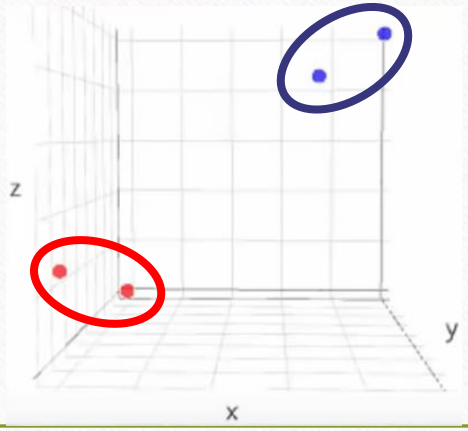


Composants du nœud de la sortie

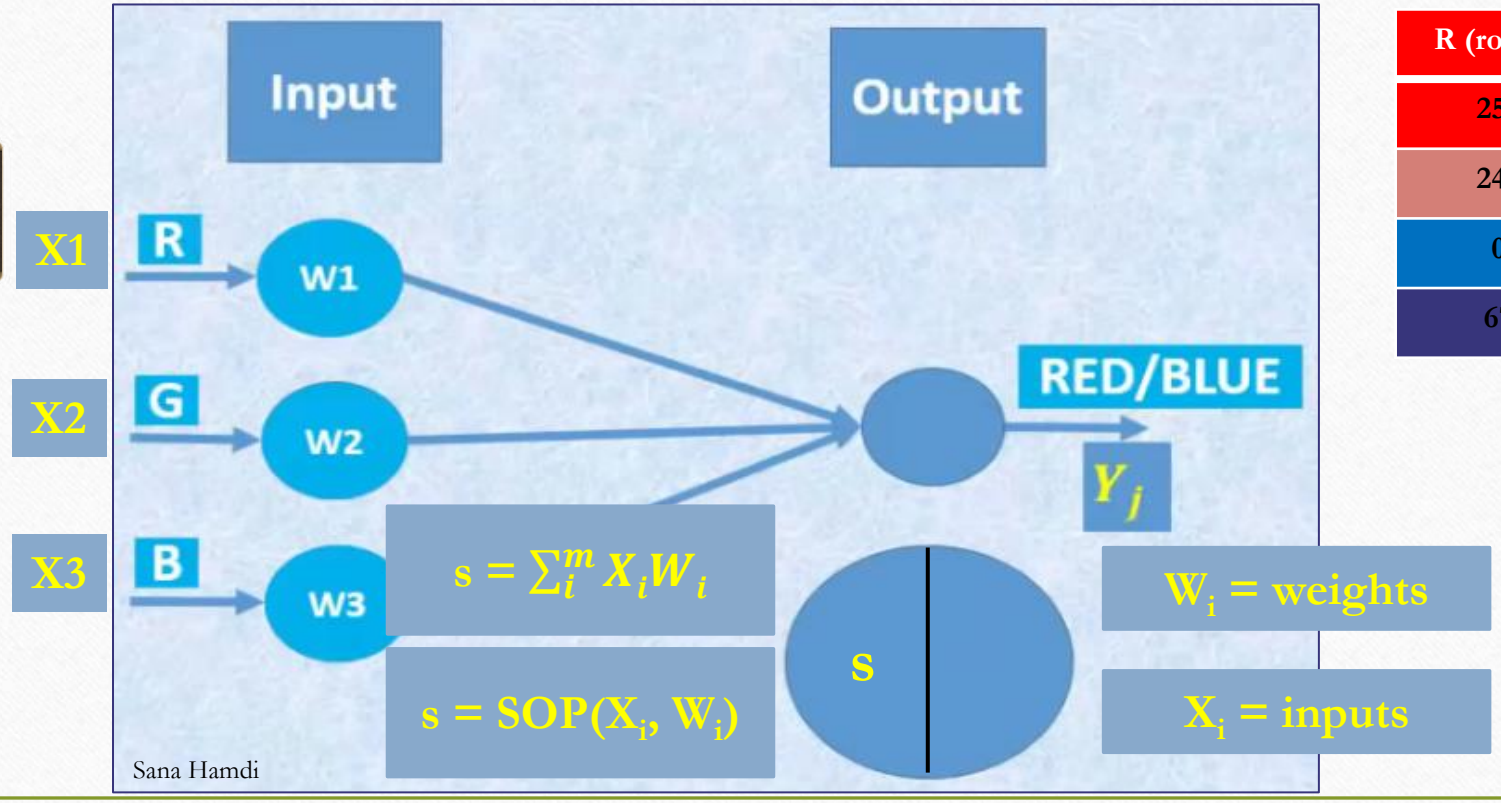


R (rouge)	G (vert)	B (bleu)	Classe
255	0	0	ROUGE
248	80	68	ROUGE
0	0	255	BLEU
67	15	210	BLEU

?

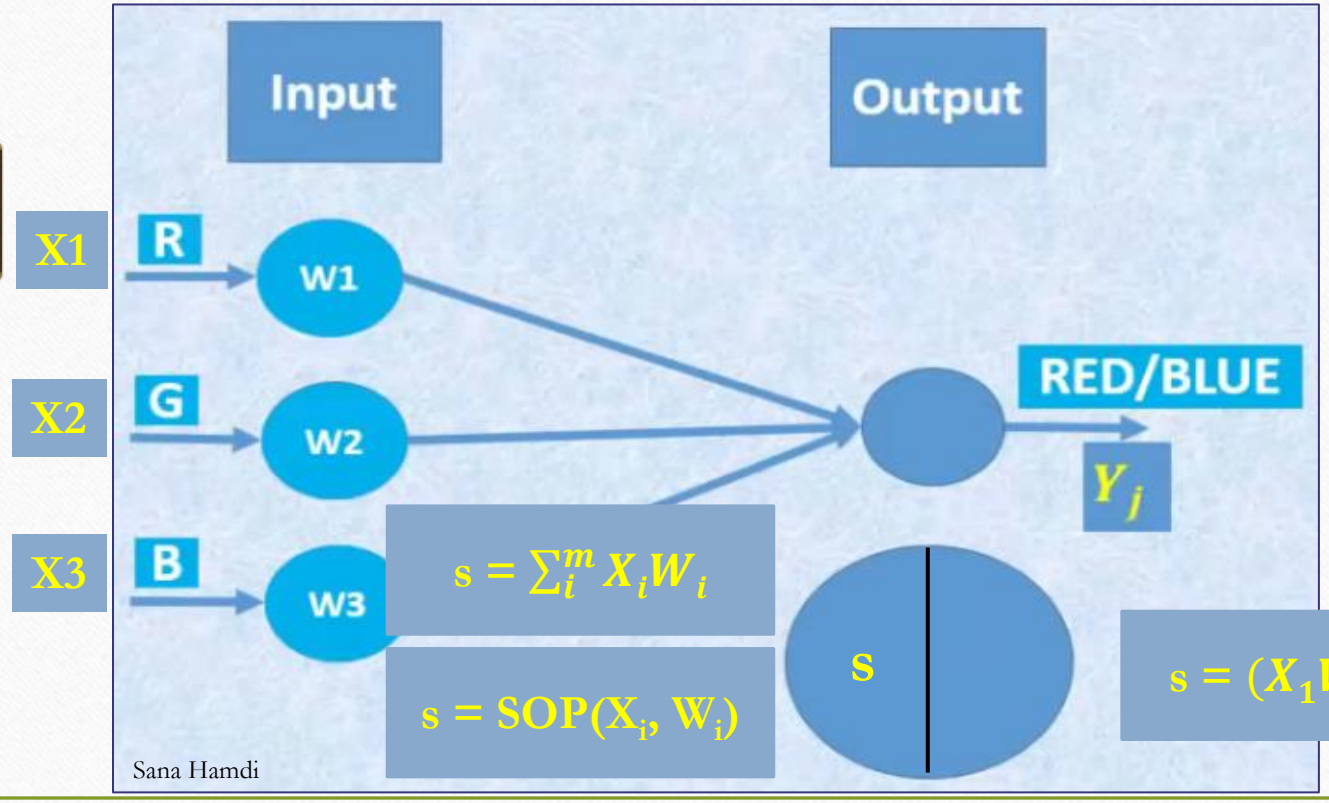


Composants du nœud de la sortie: somme pondérée



R (rouge)	G (vert)	B (bleu)	Classe
255	0	0	ROUGE
248	80	68	ROUGE
0	0	255	BLEU
67	15	210	BLEU

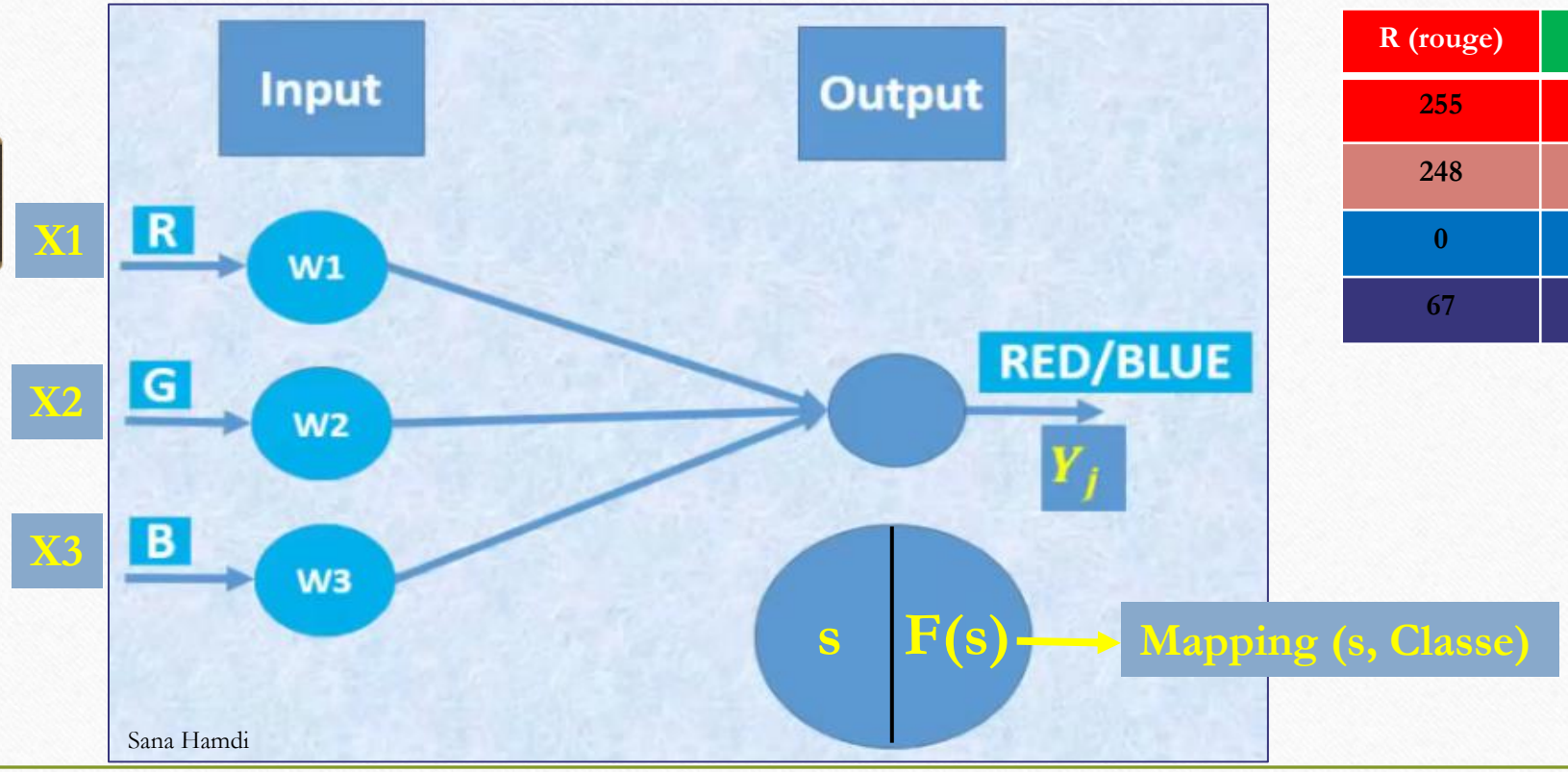
Composants du nœud de la sortie: fonction d'activation



$$s = (X_1W_1 + X_2W_2 + X_3W_3)$$

R (rouge)	G (vert)	B (bleu)	Classe
255	0	0	ROUGE
248	80	68	ROUGE
0	0	255	BLEU
67	15	210	BLEU

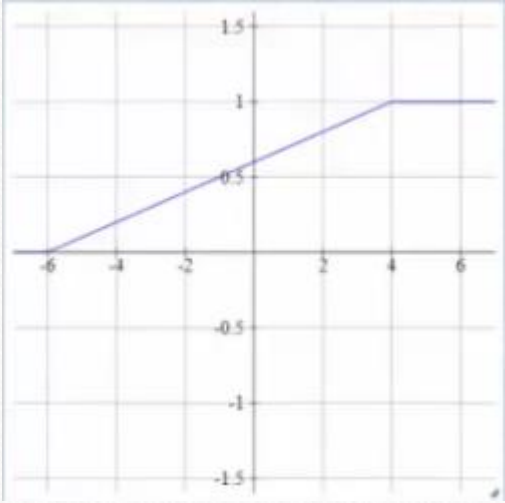
Composants du nœud de la sortie: fonction d'activation



R (rouge)	G (vert)	B (bleu)	Classe
255	0	0	ROUGE
248	80	68	ROUGE
0	0	255	BLEU
67	15	210	BLEU

Fonctions d'activation

La fonction **linéaire par morceaux**



$$f(x) = \max(0, x) + \sum_{s=1}^{S} a_i^s \max(0, -x + b_i^s)$$

La fonction **sigmoïde**

$$a_j^i = \sigma(z_j^i) = \frac{1}{1 + \exp(-z_j^i)}$$

La fonction **signe**

$$a_j^i = \sigma(z_j^i) = \begin{cases} -1 & \text{if } z_j^i < 0 \\ 1 & \text{if } z_j^i > 0 \end{cases}$$

Fonction d'activation

- Quelle fonction d'activation à choisir?

Fonction d'activation → **Output** → **Classes**

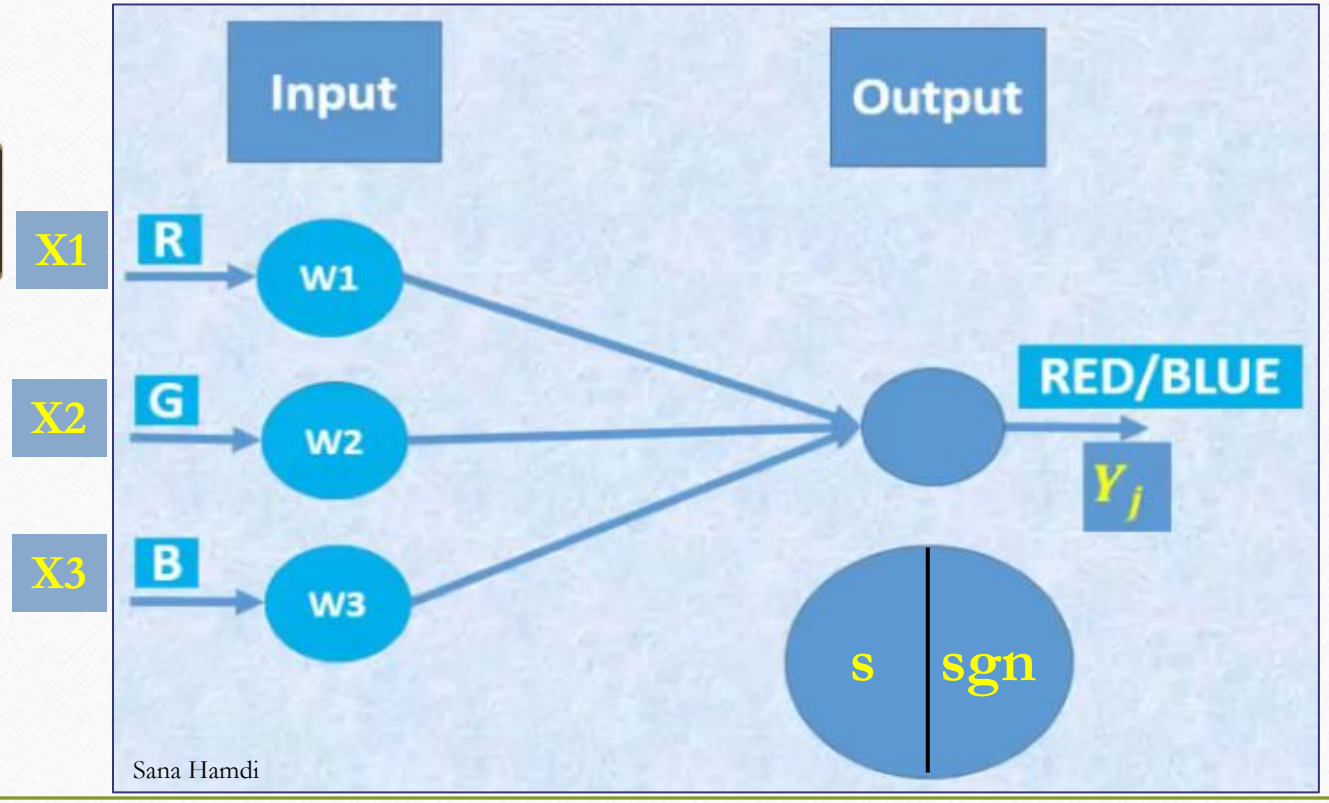
- Dans notre cas nous avons **deux** classes (ROUGE et BLEU)

→La fonction d'activation à utiliser donne seulement 2 outputs.

→**La fonction de signe**

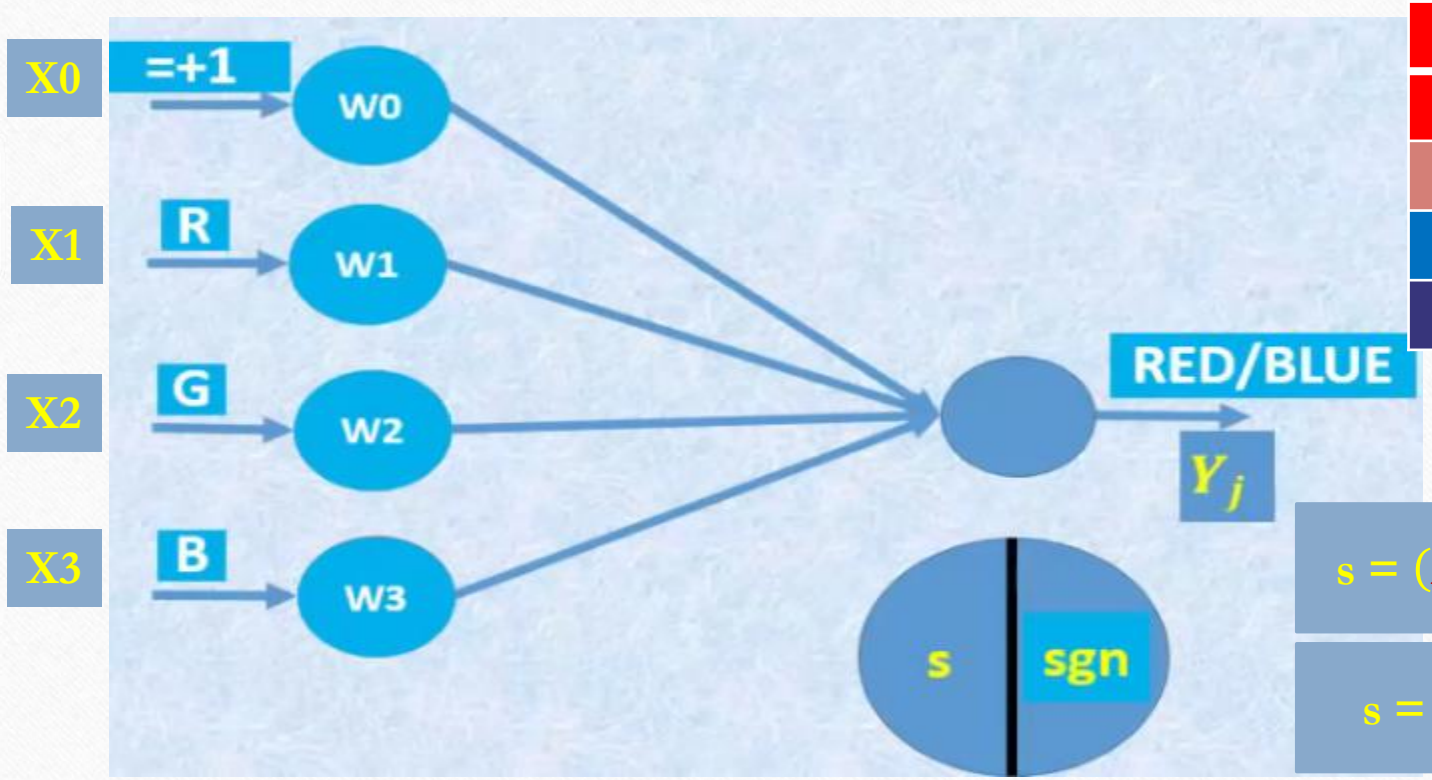
$$a_j^i = \sigma(z_j^i) = \begin{cases} -1 & \text{if } z_j^i < 0 \\ 1 & \text{if } z_j^i > 0 \end{cases}$$

Composants du nœud de la sortie



R (rouge)	G (vert)	B (bleu)	Classe
255	0	0	ROUGE
248	80	68	ROUGE
0	0	255	BLEU
67	15	210	BLEU

Le biais



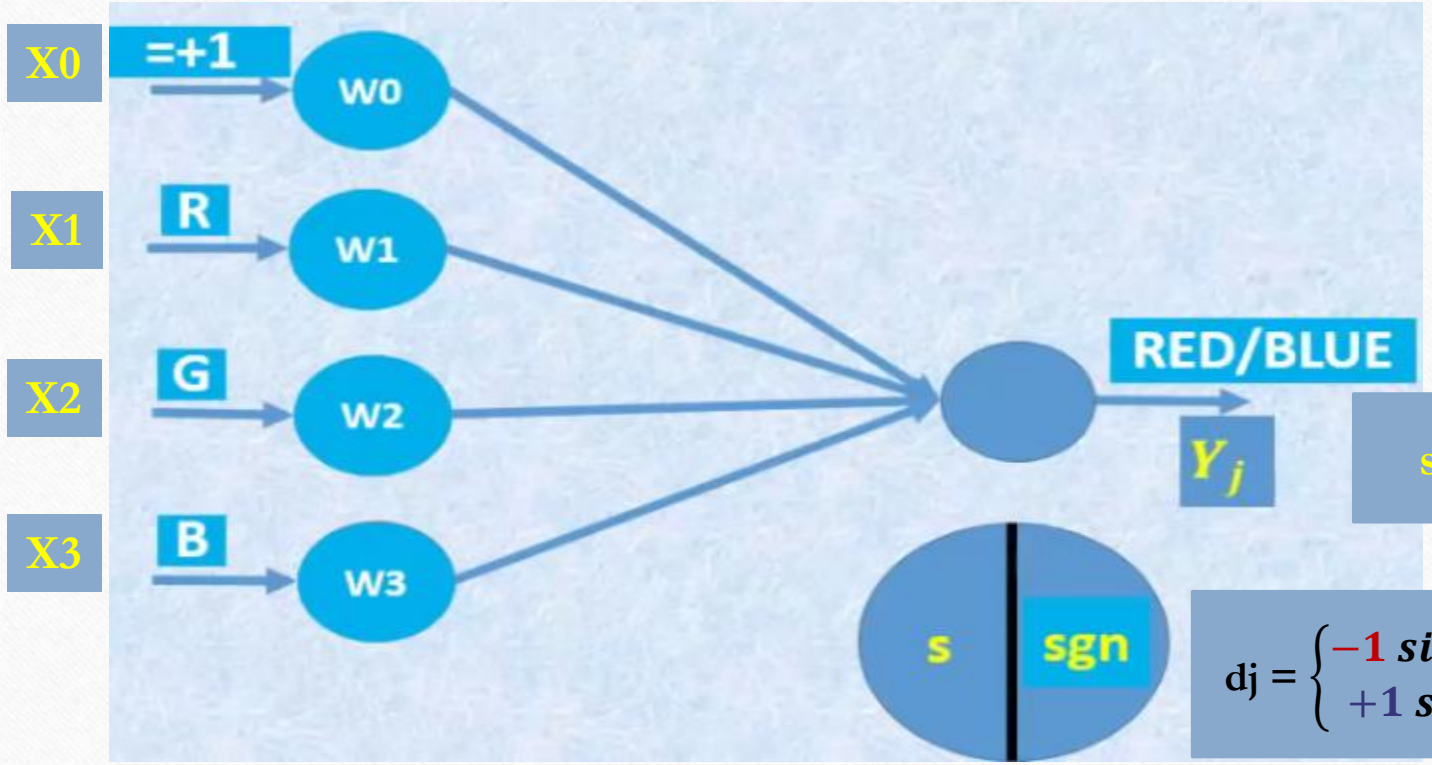
R (rouge)	G (vert)	B (bleu)	Classe
255	0	0	ROUGE
248	80	68	ROUGE
0	0	255	BLEU
67	15	210	BLEU

$$s = \sum_i^m X_i W_i + biais$$

$$s = (X_0 W_0 + X_1 W_1 + X_2 W_2 + X_3 W_3)$$

$$s = (W_0 + X_1 W_1 + X_2 W_2 + X_3 W_3)$$

La classe désirée



$$0 \leq \eta \leq 1$$

$$s = (W_0 + X_1W_1 + X_2W_2 + X_3W_3)$$

$$dj = \begin{cases} -1 \text{ si l'instance } j \text{ appartient à } C1(ROUGE) \\ +1 \text{ si l'instance } j \text{ appartient à } C2(BLEU) \end{cases}$$

L'algorithme de perceptron: étapes d'apprentissage

1. Initialiser aléatoirement les poids synaptiques
2. Passer les observations une à une
3. Calculer la somme des produits
4. Calculer la réponse de la fonction d'activation
5. Mise à jour des poids

Jusqu'à convergence

Mise à jour des poids

- Si la classe prédite $\mathbf{Y_j}$ de l'instance **j** , à l'itération (étape de calcul) **n**, est différente de celle désirée $\mathbf{d_j}$, alors les poids doivent être mis à jour selon cette équation:

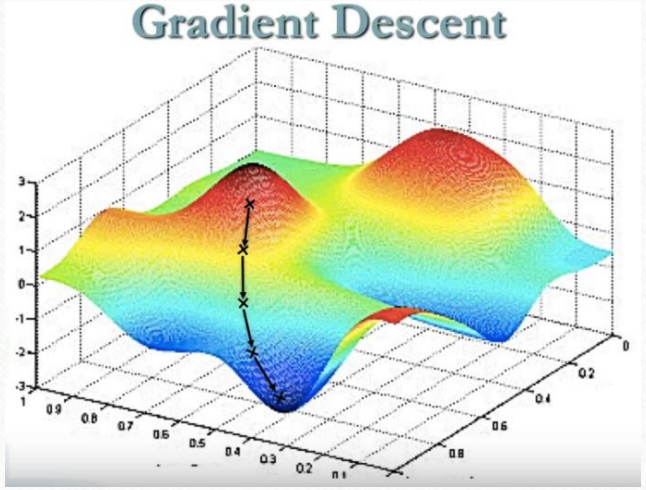
$$\mathbf{W_i(n+1) = Wi(n) + \eta[d(n) - Y(n)]X_i(n)}$$

Avec

$\mathbf{i} = [\mathbf{0}, \dots, \boldsymbol{m}]$ et m est le nombre d'entrées (attributs)

Taux d’apprentissage

- Le taux d'apprentissage est un hyper-paramètre qui présente la rapidité de la réponse du réseau aux changements qu’il subisse.
- Il contrôle à quel point nous ajustons les poids de notre réseau en fonction de la descente du gradient.
- **$0 \leq \eta \leq 1$**



Itération 0

- Pour chaque itération, les paramètres du réseau doivent être connus.
- ***<u>Paramètres à l'itération n = 0:</u>***

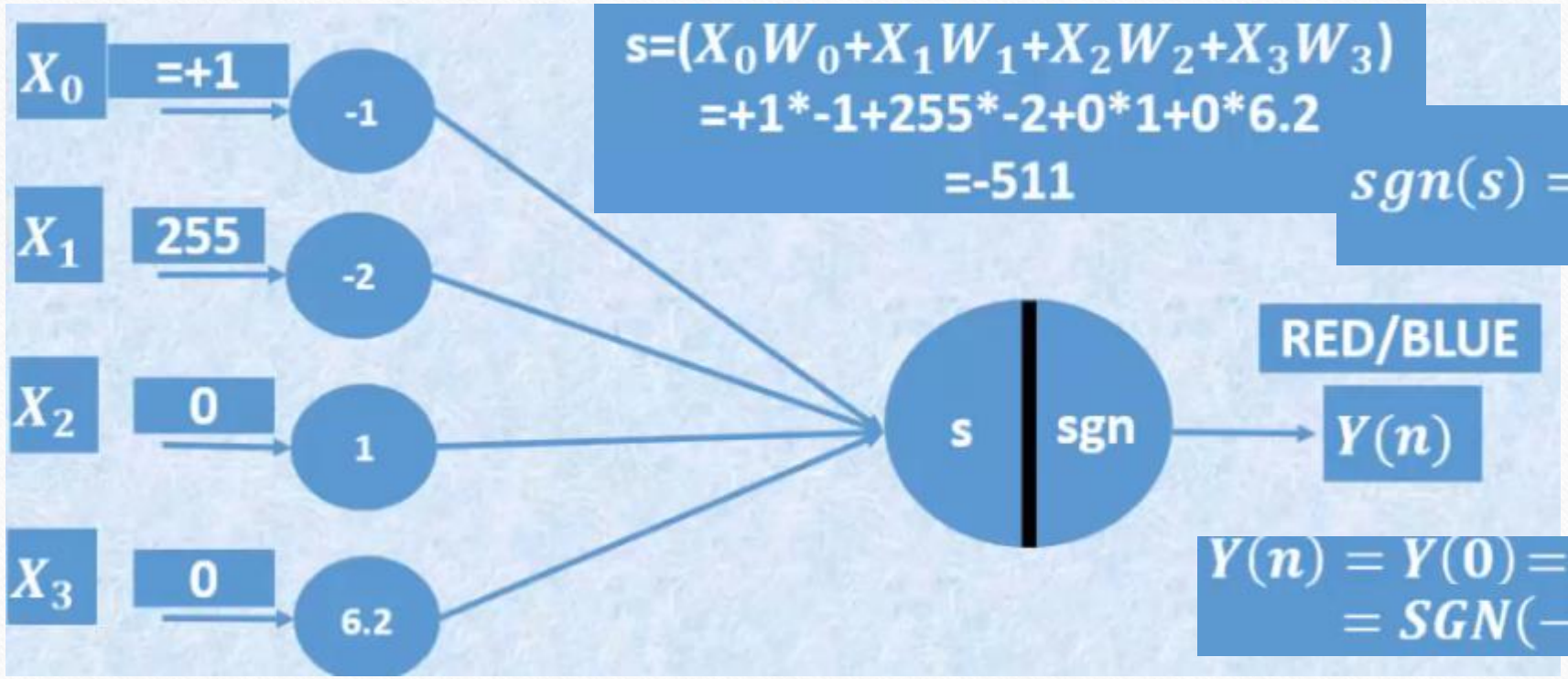
$$\eta=0{,}001$$

$$X(0)=[X_0,X_1,X_2,X_3]=[+1, 255, 0, 0]$$

$$W(0) = [W_0,W_1,W_2,W_3]=[-1, -2, 1, 6.2]$$

$$d(0)=-1$$

Itération 0



$$s=(X_0W_0+X_1W_1+X_2W_2+X_3W_3)$$
$$=+1*-1+255*-2+0*1+0*6.2$$
$$=-511$$

$$sgn(s)=\begin{cases}+1, s\geq 0\\ -1, s<0\end{cases}$$

$$Y(n)=Y(0)=SGN(s)$$
$$=SGN(-511)=-1$$

Itération 0

Y(0) = -1
d(0) = -1
Y(0) = d(0)
Poids corrects
Pas de mise à jour

Itération 1

- ***<u>Paramètres à l'itération n = 1:</u>***

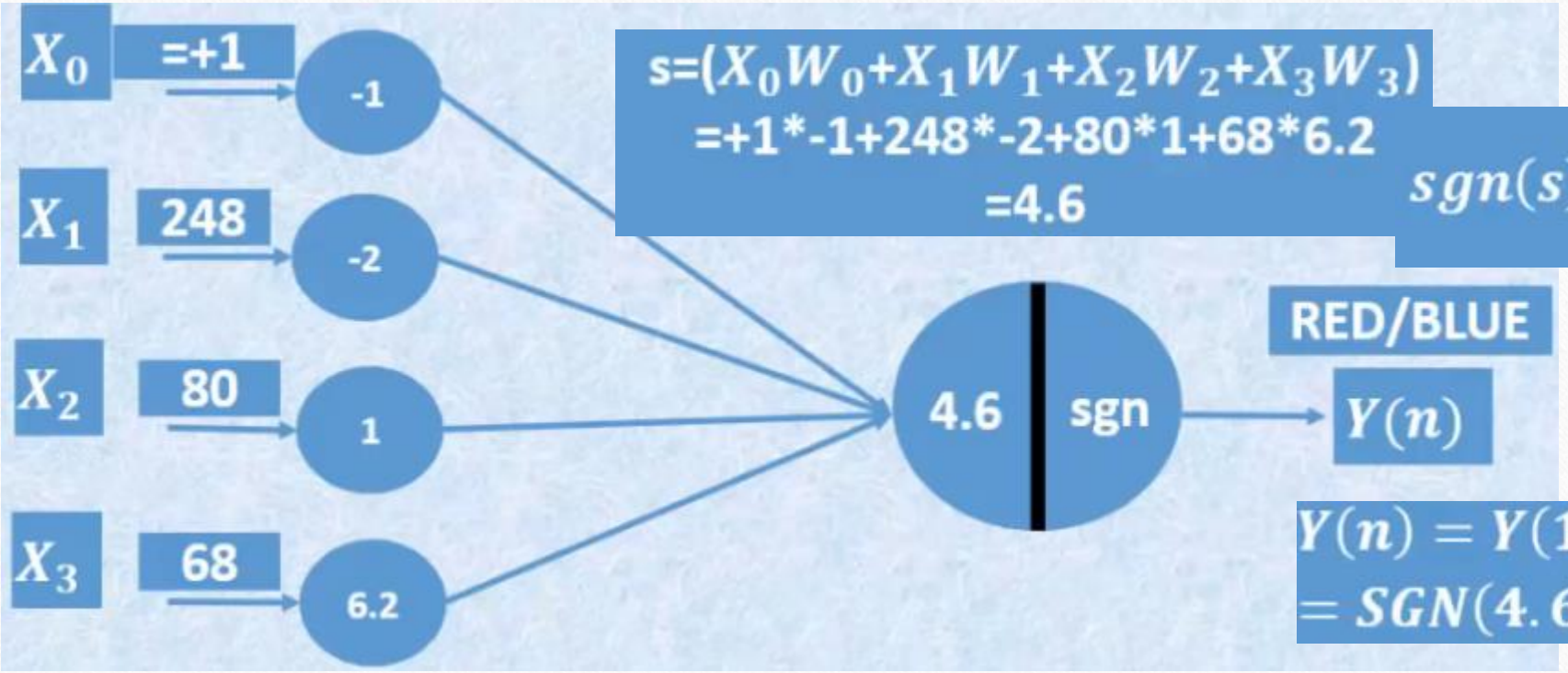
$$\eta=0{,}001$$

$$X(1)=[+1, 248, 80, 68]$$

$$W(1)=[-1, -2, 1, 6.2]$$

$$d(0)=-1$$

Itération 1

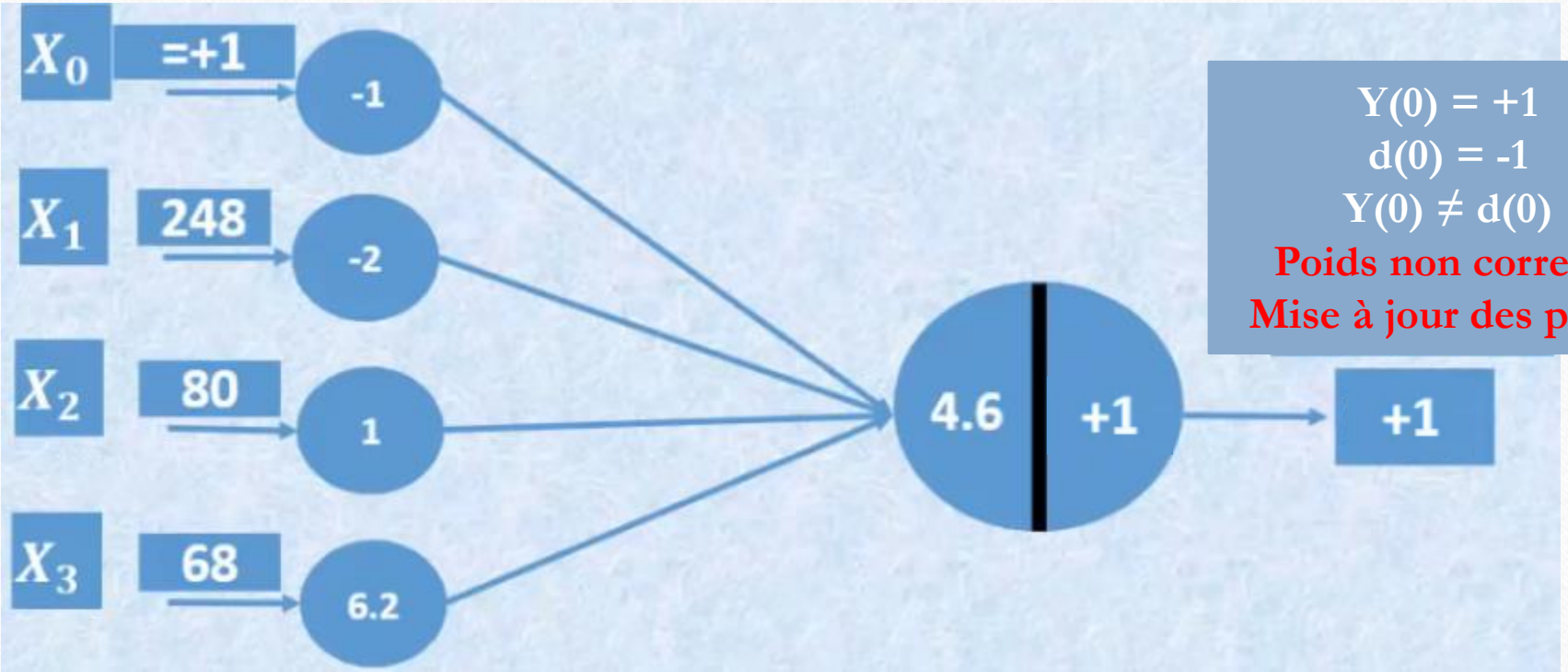


$$s=(X_0W_0+X_1W_1+X_2W_2+X_3W_3)$$
$$=+1*-1+248*-2+80*1+68*6.2$$
$$=4.6$$

$$sgn(s)=\begin{cases}+1, s\geq 0\\ -1, s<0\end{cases}$$

$$Y(n)=Y(1)=SGN(s)$$
$$=SGN(4.6)=+1$$

Itération 1



Y(0) = +1
d(0) = -1
Y(0) ≠ d(0)
Poids non corrects
Mise à jour des poids

Mise à jour des poids pour l'itération2

- Selon cette équation:

$$\mathbf{W_i(n+1) = Wi(n) + \eta[d(n) - Y(n)]Xi(n)}$$

- Avec n =1, on aura:

$$\mathbf{W(2) = W(1) + \eta[d(1) - Y(1)]X(1)}$$

$$\mathbf{W(2) = [-1, -2, 1, 6.2] + 0.001[-1-(+1)][+1, 248, 80, 68]}$$

$$\mathbf{W(2) = [-1, -2, 1, 6.2] + 0.001[-2][+1, 248, 80, 68]}$$

$$\mathbf{W(2) = [-1, -2, 1, 6.2] + [-0.002][+1, 248, 80, 68]}$$

$$\mathbf{W(2) = [-1, -2, 1, 6.2] + [-0.002, -0.496, -0.\mathit{16}, -0{,}136]}$$

$$\mathbf{W(2) = [-1.002, -2.496, 0.84, 6.064]}$$

Itération 2

- ***<u>Paramètres à l'itération n = 2:</u>***

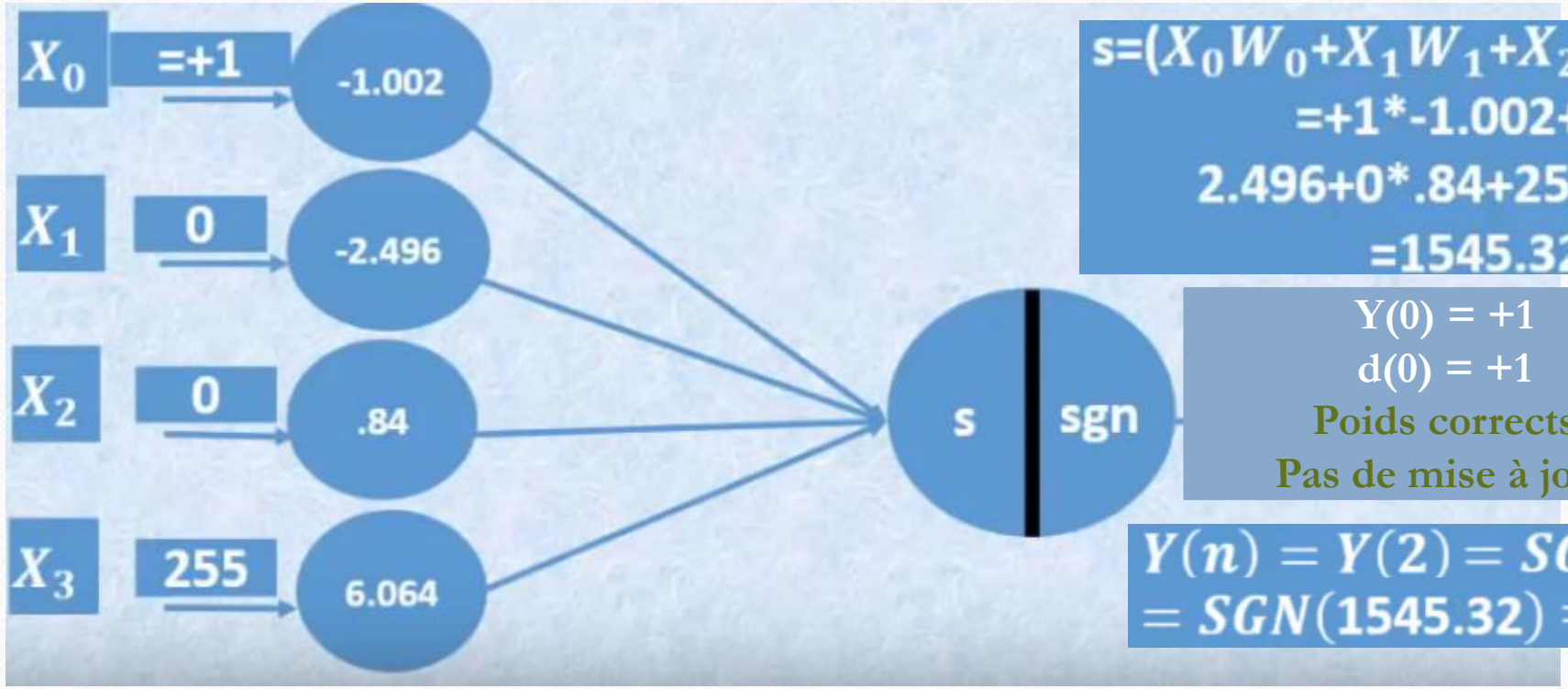
$$\eta=0{,}001$$

$$X(2)=[+1, 0, 0, 255]$$

$$\mathbf{W}(2) = [-1.002, -2.496, 0.84, 6.064]$$

$$d(0)=+1$$

Itération 2



$$s=(X_0W_0+X_1W_1+X_2W_2+X_3W_3)$$
$$=+1*-1.002+0*-2.496+0*.84+255*6.064$$
$$=1545.32$$

Y(0) = +1
d(0) = +1
Poids corrects
Pas de mise à jour

$$Y(n) = Y(2) = SGN(s) = SGN(1545.32) = +1$$

Itération 3

- ***<u>Paramètres à l'itération n = 3:</u>***

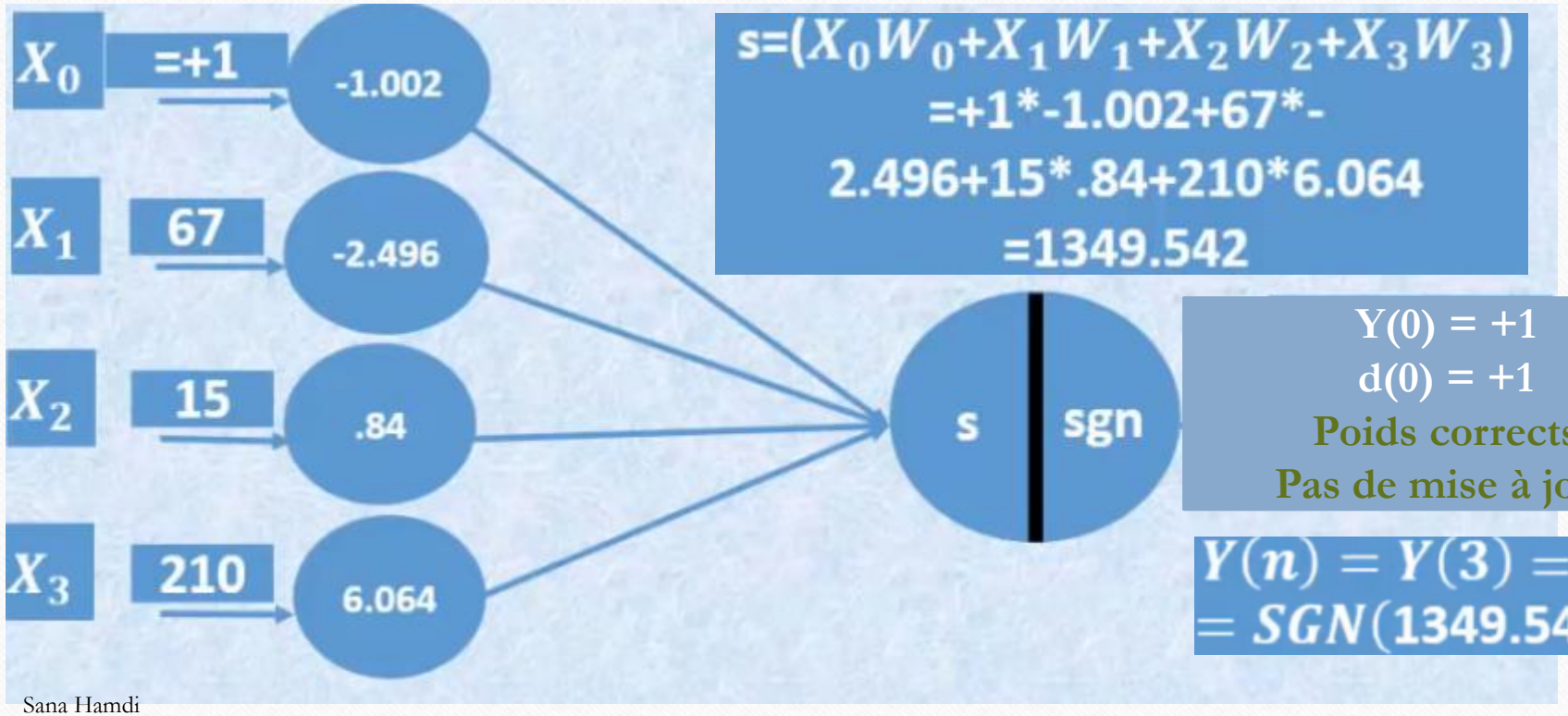
$$\eta=0{,}001$$

$$X(3)=[+1, 67, 15, 210]$$

$$\mathbf{W}(3) = [-1.002, -2.496, 0.84, 6.064]$$

$$d(0)=+1$$

Itération 3



$s=(X_0W_0+X_1W_1+X_2W_2+X_3W_3)$
$=+1*-1.002+67*-2.496+15*.84+210*6.064$
$=1349.542$

Y(0) = +1
d(0) = +1
Poids corrects
Pas de mise à jour

$Y(n) = Y(3) = SGN(s)$
$= SGN(1349.542) = +1$

Itération 4

- On n'a pas encore terminé
- On doit entrainer le modèle avec les 2 premières instances en utilisant les nouvelles valeurs des poids.
- ***<u>Paramètres à l'itération n = 4:</u>***

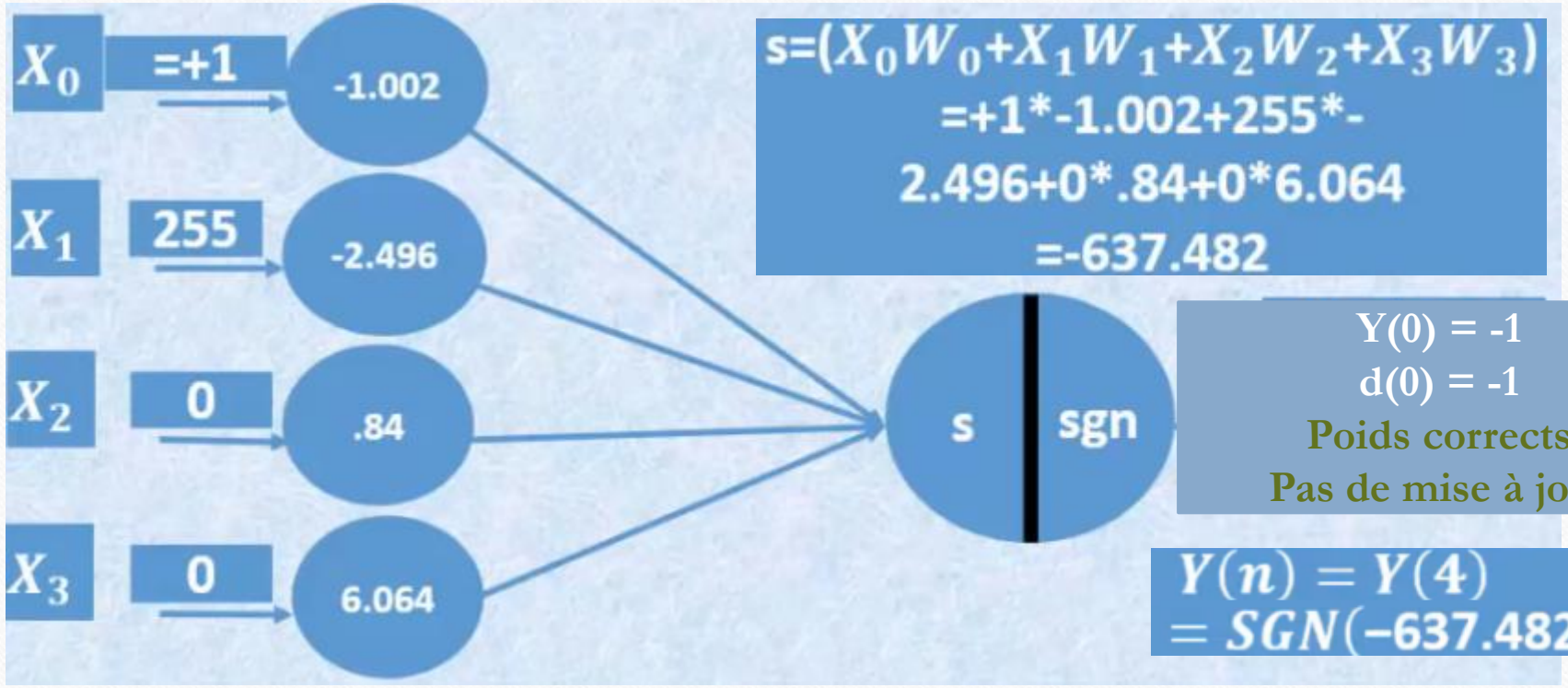
$$\eta=0{,}001$$

$$X(4)=[+1, 255, 0, 0]$$

$$\mathbf{W}(4) = [-1.002, -2.496, 0.84, 6.064]$$

$$d(0)=-1$$

Itération 4



$s=(X_0W_0+X_1W_1+X_2W_2+X_3W_3)$
$=+1*-1.002+255*-2.496+0*.84+0*6.064$
$=-637.482$

Y(0) = -1
d(0) = -1
Poids corrects
Pas de mise à jour

$Y(n) = Y(4) = SGN(-637.482) = -1$

Itération 5

- ***<u>Paramètres à l'itération n = 5:</u>***

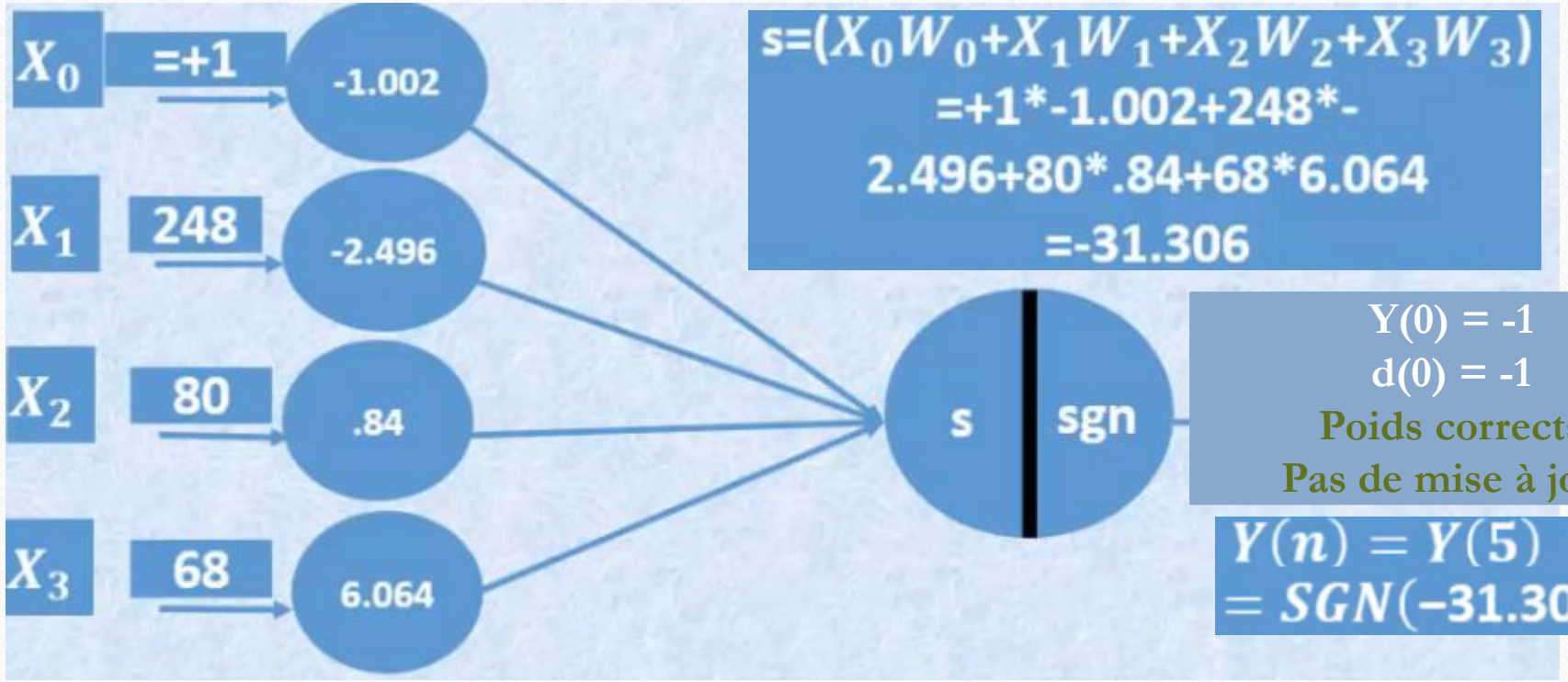
$$\eta=0{,}001$$

$$X(5)=[+1, 248, 80, 68]$$

$$\mathbf{W}(\mathbf{5}) = [-1.002, -2.496, 0.84, 6.064]$$

$$d(0)=-1$$

Itération 5



$$s=(X_0W_0+X_1W_1+X_2W_2+X_3W_3)$$
$$=+1*-1.002+248*-2.496+80*.84+68*6.064$$
$$=-31.306$$

Y(0) = -1
d(0) = -1
Poids corrects
Pas de mise à jour

$$Y(n) = Y(5) = SGN(-31.306) = -1$$

Prédiction

- Après entraîner notre réseau de neurones avec toutes les entrées en utilisant les poids mis à jour, on peut conclure que ces poids sont justes pour ce réseau et ce modèle d'apprentissage.
- Après la phase d'apprentissage, on peut utiliser le modèle pour prédire la classe d'une nouvelle instance,
- Quelle est la classe (ROUGE ou BLEU) de l'instance :

R=150, G=100, B=180